



**SISTEMA DE GESTIÓN DE BIBLIOTECA IMPLEMENTADO EN JAVA:
APLICACIÓN PRÁCTICA DE ESTRUCTURAS DE DATOS**

**JOSE ALEJANDRO MUÑOZ LAMADRID
2410180077**

**OSCAR DAVID FLEREZ CASTRO
2410180042**

**JORGE BUSTOS
JUAN JOSE PUELLO BELTRÁN**

**FUNDACION UNIVERSITARIA COLOMBO INTERNACIONAL.
TECNOLOGIA EN DESARROLLO DE SOFTWARE
SEMESTRE IV
CARTAGENA DE INDIAS DT y C.
2020.**

TABLA DE CONTENIDO

	pág.
RESUMEN Y PALABRAS CLAVES	3
ABSTRACT	3
1. INTRODUCCIÓN	5
2. PLANTEAMIENTO DEL PROBLEMA	6
3. JUSTIFICACIÓN	8
4. OBJETIVOS	10
4.1 OBJETIVO GENERAL	10
4.2 OBJETIVOS ESPECIFICOS	10
5. MARCO REFERENCIAL	12
5.1 MARCO TEÓRICO	12
5.2 MARCO HISTORICO	16
5.3 MARCO LEGAL	18
6. METODOLOGÍA	21
7. CONCLUSIONES	28
8. REFERENCIAS BIBLIOGRAFICAS	31
9. ANEXOS	33

RESUMEN Y PALABRAS CLAVES

El presente documento describe el desarrollo de un Sistema de Gestión de Biblioteca implementado en Java como proyecto de aula para el programa de Tecnología en Desarrollo de Software de Uicolombo. El proyecto aborda la necesidad de gestionar de manera eficiente el préstamo, devolución y reserva de libros en una biblioteca, mediante la implementación de estructuras de datos fundamentales como árboles binarios de búsqueda, listas enlazadas, pilas y colas.

El sistema desarrollado permite la administración completa de usuarios, libros, préstamos y reservas, con persistencia de datos mediante archivos de texto plano. Se implementó tanto una interfaz de consola como una interfaz gráfica utilizando JavaFX, logrando un sistema funcional y completo que aplica los conocimientos adquiridos en el curso de Estructuras de Datos.

Los resultados obtenidos demuestran la efectividad de las estructuras de datos en la resolución de problemas reales de gestión de información, con tiempos de búsqueda logarítmicos para libros (usando árboles binarios), gestión eficiente de colas de espera para reservas, y manejo ordenado del historial de préstamos mediante pilas.

Palabras clave: Estructuras de datos, Java, Sistema de biblioteca, Árboles binarios, Listas enlazadas, JavaFX, Gestión de préstamos.

ABSTRACT

This document describes the development of a Library Management System implemented in Java as a classroom project for the Software Development Technology program at Uicolombo. The project addresses the need to efficiently manage the lending, return, and reservation of books in a library through the implementation of fundamental data structures such as binary search trees, linked lists, stacks, and queues.

The developed system allows complete management of users, books, loans, and reservations, with data persistence through plain text files. Both a console interface and a graphical interface using JavaFX were implemented, achieving a functional and complete system that applies the knowledge acquired in the Data Structures course.

The results obtained demonstrate the effectiveness of data structures in solving real information management problems, with logarithmic search times for books (using binary trees), efficient management of waiting queues for reservations, and orderly handling of loan history through stacks.

Keywords: Data structures, Java, Library system, Binary trees, Linked lists, JavaFX, Loan management.

1. INTRODUCCIÓN

En la actualidad, la gestión eficiente de información es fundamental para cualquier organización, y las bibliotecas no son la excepción. El manejo manual de préstamos, devoluciones y reservas de libros presenta múltiples desafíos: pérdida de información, inconsistencias en los registros, dificultad para generar reportes, y tiempo excesivo en operaciones básicas de búsqueda.

El presente proyecto surge como respuesta a la necesidad de aplicar los conocimientos teóricos sobre estructuras de datos en un contexto práctico y real. Se desarrolló un Sistema de Gestión de Biblioteca que implementa estructuras de datos fundamentales aprendidas durante el curso, demostrando cómo la teoría se traduce en soluciones concretas para problemas del mundo real.

Este documento describe el proceso completo de desarrollo del sistema, desde la identificación del problema hasta la implementación y prueba de la solución. El sistema implementado permite gestionar libros, usuarios, préstamos y reservas de manera eficiente, utilizando árboles binarios de búsqueda para almacenar libros, listas enlazadas para usuarios, pilas para el historial de operaciones, y colas para el manejo de reservas.

El alcance del proyecto incluye operaciones CRUD (Crear, Leer, Actualizar, Eliminar) completas para todas las entidades del sistema, generación de reportes estadísticos, manejo de multas por retraso en devoluciones, sistema de reservas cuando los libros no están disponibles, y persistencia de datos mediante archivos de texto plano. Se desarrollaron dos interfaces: una de consola para demostrar la lógica de negocio y otra gráfica con JavaFX para mejorar la experiencia del usuario.

2. PLANTEAMIENTO DEL PROBLEMA

Las bibliotecas, tanto académicas como públicas, enfrentan constantes desafíos en la gestión de su inventario de libros y el control de préstamos a usuarios. En instituciones educativas como Unicolombo, donde cientos de estudiantes requieren acceso simultáneo a recursos bibliográficos, la necesidad de un sistema eficiente se vuelve crítica.

Situación Actual

Actualmente, muchas bibliotecas pequeñas o departamentales aún operan con sistemas manuales o semi-automatizados que presentan limitaciones significativas. Los registros en papel o en hojas de cálculo simples no permiten búsquedas rápidas, no validan automáticamente la disponibilidad de libros, no calculan multas por retraso, y no mantienen un historial detallado de transacciones.

Cuando un estudiante necesita consultar un libro, el proceso manual implica: buscar en un catálogo físico o digital básico, verificar manualmente la disponibilidad, registrar el préstamo en un cuaderno o archivo Excel, calcular manualmente la fecha de devolución, y recordar hacer seguimiento a las devoluciones atrasadas. Este proceso es propenso a errores humanos, consume tiempo valioso del personal, dificulta la generación de estadísticas, y no escala cuando aumenta el número de usuarios o libros.

Elementos del Problema

Los principales elementos que componen este problema son:

Datos a gestionar: información de libros (título, autor, ISBN, categoría, año, disponibilidad), información de usuarios (identificación, nombre, email, límite de

préstamos, multas acumuladas), registros de préstamos (fechas, estado, renovaciones), y registros de reservas (cola de espera para libros prestados).

Operaciones requeridas: búsqueda rápida de libros por diferentes criterios (ID, autor, categoría), validación de disponibilidad en tiempo real, cálculo automático de fechas de devolución y multas, gestión de colas de espera para libros no disponibles, y generación de reportes estadísticos.

Restricciones: un libro solo puede ser prestado a un usuario a la vez, cada usuario tiene un límite de préstamos simultáneos, las multas deben calcularse automáticamente por días de retraso, y las reservas deben procesarse en orden de llegada (FIFO).

Pregunta de Investigación

¿Cómo implementar un sistema de gestión de biblioteca que utilice estructuras de datos eficientes para garantizar tiempos de respuesta óptimos en operaciones de búsqueda, inserción y eliminación, mientras mantiene la integridad de la información y permite la persistencia de datos?

3. JUSTIFICACIÓN

El desarrollo de este Sistema de Gestión de Biblioteca se justifica desde múltiples perspectivas: académica, técnica, práctica y social.

Justificación Académica

Desde el punto de vista académico, este proyecto permite consolidar y aplicar los conocimientos adquiridos durante el curso de Estructuras de Datos. La teoría sobre árboles binarios, listas enlazadas, pilas y colas cobra sentido cuando se implementa en un sistema real que resuelve problemas concretos. El proyecto exige comprender no solo cómo funcionan estas estructuras, sino también cuándo y por qué usar cada una, desarrollando criterio técnico para la selección de estructuras de datos según los requisitos del problema.

Este proyecto de aula para Unicolombo, en el programa de Tecnología en Desarrollo de Software, representa una oportunidad para demostrar competencias en programación orientada a objetos, diseño de software, manejo de persistencia de datos, desarrollo de interfaces de usuario, y documentación técnica.

Justificación Técnica

La elección de estructuras de datos apropiadas tiene un impacto directo en el rendimiento del sistema:

Árboles binarios de búsqueda para libros: garantizan búsquedas en tiempo $O(\log n)$ en promedio, comparado con $O(n)$ de búsqueda lineal en arrays. Para una biblioteca con 1000 libros, esto significa encontrar un libro en aproximadamente 10 comparaciones en lugar de 500 en promedio.

Listas enlazadas para usuarios: permiten inserción y eliminación eficientes sin necesidad de reorganizar toda la estructura, ideal cuando la cantidad de usuarios varía frecuentemente.

Colas para reservas: garantizan justicia en el orden de atención (FIFO - First In, First Out), asegurando que quien reserva primero recibe el libro primero cuando esté disponible.

Pilas para historial: permiten acceso rápido a las operaciones más recientes y facilitan funcionalidades como "deshacer" o consultar últimas transacciones.

Justificación Práctica

El sistema desarrollado proporciona beneficios concretos para bibliotecas reales:

Ahorro de tiempo: búsquedas instantáneas en lugar de revisión manual de registros.

Reducción de errores: validaciones automáticas previenen préstamos duplicados o exceso de límites.

Automatización de cálculos: fechas de devolución y multas se calculan automáticamente.

Generación de reportes: estadísticas instantáneas sobre libros más prestados, usuarios más activos, estado financiero de multas.

Persistencia de datos: la información se guarda automáticamente y sobrevive al cierre del programa.

4. OBJETIVOS

4.1 OBJETIVO GENERAL

Desarrollar e implementar un sistema de gestión de biblioteca utilizando estructuras de datos fundamentales en Java, que permita administrar eficientemente libros, usuarios, préstamos y reservas, garantizando tiempos de respuesta óptimos mediante la aplicación de árboles binarios de búsqueda, listas enlazadas, pilas y colas, para demostrar la aplicación práctica de los conceptos teóricos de estructuras de datos en la resolución de problemas reales.

4.2 OBJETIVOS ESPECIFICOS

1. Diseñar la arquitectura del sistema definiendo las clases, estructuras de datos y relaciones entre componentes, mediante diagramas UML y especificación de interfaces, para establecer una base sólida que soporte todos los requerimientos funcionales del sistema.
2. Implementar un árbol binario de búsqueda para el almacenamiento y gestión de libros, incluyendo operaciones de inserción, búsqueda, eliminación y recorrido, para garantizar búsquedas eficientes con complejidad logarítmica.
3. Desarrollar gestores de usuarios, préstamos y reservas utilizando listas enlazadas, pilas y colas respectivamente, aplicando las operaciones características de cada estructura, para administrar correctamente las diferentes entidades del sistema.
4. Crear un sistema de persistencia de datos mediante archivos de texto plano, que permita guardar y recuperar información de libros, usuarios, préstamos y reservas, para mantener la integridad de los datos entre sesiones del programa.

5. Implementar dos interfaces de usuario (consola y gráfica con JavaFX), que expongan todas las funcionalidades del sistema de manera intuitiva, para facilitar la interacción del usuario final con el sistema.
6. Desarrollar un módulo de generación de reportes que proporcione estadísticas sobre el uso del sistema, incluyendo libros más prestados, usuarios más activos y estado financiero de multas, para apoyar la toma de decisiones administrativas.
7. Validar el correcto funcionamiento del sistema mediante pruebas exhaustivas de cada componente y del sistema integrado, para garantizar que cumple con todos los requerimientos establecidos.

5. MARCO REFERENCIAL

5.1 MARCO TEÓRICO

Estructuras de Datos

Las estructuras de datos son formas organizadas de almacenar y gestionar información en la memoria de una computadora, diseñadas para permitir acceso y modificación eficientes. La elección correcta de una estructura de datos impacta directamente en el rendimiento de un programa[1].

Árboles Binarios de Búsqueda (BST)

Un árbol binario de búsqueda es una estructura jerárquica donde cada nodo tiene como máximo dos hijos (izquierdo y derecho), cumpliendo la propiedad de orden: todos los valores del subárbol izquierdo son menores que el valor del nodo, y todos los valores del subárbol derecho son mayores[2]. Esta propiedad garantiza búsquedas eficientes.

Las operaciones fundamentales en un BST tienen complejidad temporal promedio de $O(\log n)$:

- **Inserción:** Se compara el nuevo valor con la raíz y se avanza recursivamente hacia izquierda o derecha hasta encontrar una posición vacía.
- **Búsqueda:** Similar a inserción, se compara el valor buscado con cada nodo, eliminando la mitad del espacio de búsqueda en cada paso.
- **Eliminación:** Caso complejo que requiere considerar tres situaciones: nodo hoja (sin hijos), nodo con un hijo, y nodo con dos hijos (requiere encontrar sucesor o predecesor).

En el peor caso (árbol degenerado en lista), la complejidad puede degradarse a $O(n)$. Para evitar esto, existen árboles balanceados como AVL o Rojo-Negro, aunque no fueron implementados en este proyecto[3].

Listas Enlazadas

Una lista enlazada es una colección de nodos donde cada nodo contiene un dato y una referencia (enlace) al siguiente nodo. A diferencia de los arrays, las listas enlazadas no requieren memoria contigua y permiten inserción y eliminación eficientes en cualquier posición[4].

Existen variantes:

- **Lista simplemente enlazada:** Cada nodo apunta solo al siguiente.
- **Lista doblemente enlazada:** Cada nodo apunta al siguiente y al anterior, facilitando recorridos bidireccionales.
- **Lista circular:** El último nodo apunta al primero, formando un ciclo.

Las operaciones típicas incluyen inserción al inicio $O(1)$, inserción al final $O(n)$ sin referencia a la cola, búsqueda $O(n)$, y eliminación $O(n)$ [5].

Pilas (Stacks)

Una pila es una estructura LIFO (Last In, First Out) donde el último elemento insertado es el primero en ser removido. Las operaciones principales son:

- **push:** Insertar elemento en el tope.
- **pop:** Remover y devolver elemento del tope.
- **peek:** Consultar elemento del tope sin removerlo.

Todas las operaciones tienen complejidad $O(1)$. Las pilas se usan en evaluación de expresiones, manejo de recursión, algoritmos de backtracking, y gestión de historial de operaciones[6].

Colas (Queues)

Una cola es una estructura FIFO (First In, First Out) donde el primer elemento insertado es el primero en ser removido. Las operaciones principales son:

- **enqueue:** Insertar elemento al final.
- **dequeue:** Remover y devolver elemento del frente.
- **front:** Consultar elemento del frente sin removerlo.

Las colas se implementan con arrays circulares o listas enlazadas. Se utilizan en sistemas de atención por turno, buffers de impresión, gestión de procesos en sistemas operativos, y algoritmos BFS (Breadth-First Search)[7].

Complejidad Algorítmica

La notación Big-O describe el comportamiento asintótico de un algoritmo, indicando cómo crece el tiempo de ejecución o uso de memoria en relación al tamaño de entrada n [8].

Estructura	Búsqueda	Inserción
Array ordenado	$O(\log n)$	$O(n)$
Lista enlazada	$O(n)$	$O(1)$ al inicio
BST (promedio)	$O(\log n)$	$O(\log n)$
BST (peor caso)	$O(n)$	$O(n)$
Hash Table	$O(1)$ promedio	$O(1)$ promedio

Table 1: Complejidad temporal de operaciones comunes

Para este proyecto, se priorizó el uso de BST para libros debido a la necesidad de búsquedas rápidas y mantenimiento del orden (útil para mostrar libros ordenados alfabéticamente).

Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el código en objetos que encapsulan datos y comportamiento. Los principios fundamentales son[9]:

Encapsulamiento: Ocultar detalles de implementación, exponiendo solo interfaces públicas. En el proyecto, las estructuras de datos tienen métodos privados de manipulación interna.

Herencia: Reutilización de código mediante relaciones "es-un". Por ejemplo, Admin podría heredar de Usuario si se extendiera el sistema.

Polimorfismo: Capacidad de objetos de diferentes clases de responder al mismo mensaje. Las excepciones personalizadas heredan de BibliotecaException.

Abstracción: Modelar entidades del mundo real en clases. Las clases Libro, Usuario, Prestamo abstraen conceptos del dominio de bibliotecas.

Persistencia de Datos

La persistencia se refiere a mantener datos más allá del tiempo de vida de un programa. Las opciones incluyen[10]:

- **Archivos de texto:** Simple, legible, pero ineficiente para grandes volúmenes.
- **Archivos binarios:** Más compactos y rápidos, pero no legibles por humanos.
- **Bases de datos relacionales:** SQL, ideal para datos estructurados con relaciones complejas.
- **Bases de datos NoSQL:** MongoDB, Cassandra, para datos no estructurados o alta escalabilidad.
- **Serialización:** Convertir objetos a formato almacenable (JSON, XML, Java Serializable).

Este proyecto utiliza archivos de texto plano con formato delimitado por pipe (|), priorizando simplicidad y facilidad de depuración sobre eficiencia.

Interfaz Gráfica con JavaFX

JavaFX es el framework moderno de Java para desarrollo de interfaces gráficas, reemplazando a Swing. Características principales[11]:

- **Arquitectura MVC:** Separación de vista (FXML), controlador (Java) y modelo (clases de negocio).
- **Declaración de interfaces:** FXML permite diseñar interfaces en XML, similar a HTML.
- **CSS styling:** Personalización visual mediante hojas de estilo.
- **Binding:** Sincronización automática entre propiedades de objetos y elementos UI.
- **Scene Graph:** Jerarquía de nodos que representa la interfaz visual.

El proyecto implementa controladores para cada vista (LoginController, DashboardController, LibrosView, UsuariosView, PrestamosView, ReservasView, ReportesView), siguiendo el patrón MVC.

5.2 MARCO HISTORICO

Evolución de los Sistemas de Gestión de Bibliotecas

Los sistemas automatizados de bibliotecas han evolucionado significativamente desde los años 1960. El primer sistema integrado de biblioteca, NOTIS (Northwestern Online Total Integrated System), fue desarrollado en 1970 en Northwestern University[12].

En los años 1980, surgieron sistemas comerciales como VTLS (Virginia Tech Library System) y Dynix, que popularizaron la automatización bibliotecaria en instituciones medianas y pequeñas. Estos sistemas inicialmente solo manejaban catalogación y circulación, sin funcionalidades avanzadas[13].

Durante los años 1990, con la expansión de Internet, aparecieron sistemas web-based que permitían consulta del catálogo desde cualquier ubicación. OPAC (Online Public Access Catalog) se convirtió en estándar de la industria[14].

En la década de 2000, los sistemas evolucionaron hacia Integrated Library Systems (ILS) completos, integrando adquisiciones, catalogación, circulación, y gestión de recursos electrónicos. Ejemplos notables son Koha (primer ILS de código abierto, 1999), Evergreen (2006), y soluciones comerciales como Sierra y Alma[15].

Actualmente, los sistemas tienden hacia arquitecturas de servicios en la nube, integración con plataformas de aprendizaje (LMS), análisis de datos de uso, y experiencias móviles. Discovery services como Summon y Primo permiten búsqueda unificada en múltiples fuentes[16].

Estructuras de Datos en Ciencias de la Computación

El estudio formal de estructuras de datos se consolidó en los años 1960. Donald Knuth publicó "The Art of Computer Programming" (1968), que se convirtió en referencia fundamental sobre algoritmos y estructuras de datos[17].

Los árboles binarios de búsqueda fueron descritos formalmente por Adel'son-Vel'sky y Landis en 1962, cuando propusieron los árboles AVL, primera estructura de árbol balanceado[18]. Posteriormente, Rudolf Bayer inventó los árboles B en 1971, optimizados para sistemas de almacenamiento secundario[19].

El concepto de tipo abstracto de datos (ADT) fue formalizado en los años 1970, permitiendo separar la interfaz de una estructura de su implementación. Esto facilitó el desarrollo de bibliotecas estándar de estructuras de datos[20].

En 1998, Java introdujo el Collections Framework (JDK 1.2), estandarizando implementaciones de listas, conjuntos, mapas y colas. Este framework democratizó el uso de estructuras de datos eficientes en el desarrollo de aplicaciones empresariales[21].

Proyectos Similares en el Ámbito Académico

Diversos estudios han abordado el desarrollo de sistemas de gestión bibliotecaria con enfoque académico:

García y Rodríguez (2020) desarrollaron un sistema de biblioteca universitaria usando Java EE y MySQL, enfocándose en arquitectura de tres capas y patrones de diseño empresarial. Reportaron mejoras del 60% en tiempo de procesamiento de préstamos comparado con el sistema manual previo[22].

Martínez et al. (2019) implementaron un sistema de biblioteca digital para una institución educativa colombiana, utilizando tecnologías web (Spring Boot, Angular, PostgreSQL). El estudio destacó la importancia de interfaces intuitivas para usuarios no técnicos[23].

Chen y Wang (2021) compararon diferentes estructuras de datos para sistemas de recomendación de libros, concluyendo que árboles de decisión y grafos eran más efectivos que estructuras lineales para modelar preferencias de usuarios[24].

5.3 MARCO LEGAL

En Colombia, el desarrollo de sistemas de información y gestión de datos está regulado por normativas que protegen la privacidad y propiedad intelectual.

Ley 1581 de 2012 - Protección de Datos Personales

Esta ley establece disposiciones generales para la protección de datos personales en Colombia. Los artículos relevantes para este proyecto son:

Artículo 4: Define dato personal como "cualquier información vinculada o que pueda asociarse a una o varias personas naturales determinadas o determinables". El sistema de biblioteca almacena nombres, correos electrónicos e identificaciones de usuarios, constituyendo datos personales.

Artículo 5: Establece principios para el tratamiento de datos personales, incluyendo principio de finalidad (los datos se recolectan con propósito específico: gestión de préstamos), principio de seguridad (deben implementarse medidas técnicas para proteger los datos), y principio de confidencialidad (quienes accedan a los datos tienen deber de guardar reserva)[25].

Artículo 17: Deberes de los responsables del tratamiento, incluyendo garantizar que la información sea veraz, completa y actualizada, y conservar la información bajo condiciones de seguridad.

Para este proyecto académico, los datos son ficticios, pero en una implementación real sería necesario obtener consentimiento informado de usuarios, implementar cifrado de datos sensibles, y establecer políticas de retención y eliminación de datos.

Ley 23 de 1982 - Derechos de Autor

Regula los derechos de autor en Colombia. El **Artículo 2** establece que las obras literarias están protegidas, lo cual es relevante para un sistema de biblioteca que gestiona libros como activos intelectuales[26].

Artículo 37: Establece que las bibliotecas pueden reproducir obras para conservación sin fines de lucro. Un sistema de gestión debe garantizar que los préstamos respeten los términos de licencia de cada obra.

Decreto 1377 de 2013

Reglamenta parcialmente la Ley 1581 de 2012. El **Artículo 10** establece que las bases de datos deben estar inscritas en el Registro Nacional de Bases de Datos (RNBD), aunque se exceptúan bases de datos mantenidas en ámbito personal o doméstico[27].

Para una implementación institucional del sistema, sería necesario registrar la base de datos de usuarios y documentar las políticas de tratamiento de información.

Normas Técnicas de Calidad de Software

Aunque no son leyes, existen estándares internacionales adoptados en Colombia para calidad de software:

ISO/IEC 25010: Define modelo de calidad de software con características como funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad[28].

El proyecto aplica estas consideraciones implementando validación de datos (funcionalidad), manejo de excepciones (confiabilidad), interfaces intuitivas (usabilidad), y estructuras de datos eficientes (eficiencia).

6. METODOLOGÍA

Enfoque de Investigación

Este proyecto adoptó un enfoque **mixto** que combina elementos cualitativos y cuantitativos:

Cualitativo: Análisis de requisitos funcionales, diseño de arquitectura, evaluación de usabilidad de interfaces, y documentación de decisiones de diseño.

Cuantitativo: Medición de complejidad algorítmica, comparación de tiempos de ejecución entre diferentes estructuras, conteo de operaciones, y análisis de rendimiento.

Tipo de Investigación

La investigación es de tipo **aplicada y propositiva**:

Aplicada: Aplica conocimientos teóricos de estructuras de datos a un problema práctico real (gestión de bibliotecas).

Propositiva: Propone una solución concreta materializada en un sistema de software funcional.

El proyecto tiene también componente **descriptivo** al documentar cómo se comportan diferentes estructuras de datos en escenarios específicos del dominio.

Población y Muestra

Población: Instituciones educativas con bibliotecas que requieren sistemas de gestión automatizados.

Muestra: Para validación del sistema, se utilizaron datos de prueba representativos:

- 50 libros de diferentes categorías (Programación, Matemáticas, Literatura, Historia, Ciencia)

- 20 usuarios con diferentes perfiles (límites de préstamo de 2 a 5 libros)
- 35 transacciones de préstamo en diferentes estados (activos, completados, vencidos)
- 10 reservas en cola de espera

Estos datos de prueba simulan condiciones reales de una biblioteca departamental pequeña.

Fases del Desarrollo

El proyecto se ejecutó siguiendo una metodología iterativa e incremental, organizada en seis fases:

Fase 1: Análisis y Diseño

Actividades:

- Identificación de requisitos funcionales y no funcionales
- Definición de entidades del dominio (Libro, Usuario, Préstamo, Reserva)
- Selección de estructuras de datos apropiadas para cada componente
- Diseño de diagramas de clases UML
- Especificación de interfaces y contratos

Herramientas utilizadas: Papel y lápiz para bocetos iniciales, draw.io para diagramas UML.

Resultados: Documento de especificación de requisitos, diagrama de clases completo, especificación de estructuras de datos a implementar.

Fase 2: Implementación de Estructuras de Datos

Actividades:

- Implementación de árbol binario de búsqueda genérico
- Implementación de lista enlazada con operaciones CRUD
- Implementación de pila con array dinámico
- Implementación de cola con lista enlazada
- Pruebas unitarias de cada estructura aislada

Enfoque: Desarrollo desde lo general hacia lo específico. Primero se crearon estructuras genéricas reutilizables, luego se adaptaron al dominio de bibliotecas.

Validación: Cada estructura se probó independientemente con datos de prueba antes de integrarla al sistema.

Fase 3: Implementación de Lógica de Negocio

Actividades:

- Desarrollo de clases de modelo (Libro, Usuario, Préstamo, Reserva, Admin)
- Implementación de gestores (GestorLibros, GestorUsuarios, GestorPrestamos, GestorReservas)
- Implementación de reglas de negocio (validaciones, cálculo de multas, gestión de disponibilidad)
- Creación de excepciones personalizadas para manejo de errores
- Desarrollo de módulo de generación de reportes

Validaciones implementadas:

- Usuario no puede tener más préstamos que su límite establecido
- Libro no puede ser prestado si no está disponible

- Fechas de devolución deben ser posteriores a fechas de préstamo
- ISBNs deben tener formato válido (10 o 13 dígitos)
- Emails deben tener formato válido

Fase 4: Implementación de Persistencia

Actividades:

- Diseño del formato de archivos (delimitado por pipe |)
- Implementación de métodos de serialización (objetos a texto)
- Implementación de métodos de deserialización (texto a objetos)
- Manejo de errores de lectura/escritura
- Creación de directorio de datos si no existe

Formato de persistencia:

Cada entidad se guarda en un archivo separado:

libros.txt: ID|Título|Autor|ISBN|Año|Categoría|Disponible
 usuarios.txt: ID|Nombre|Email|Límite|PréstamosActuales|Multa
 prestamos.txt: ID|IDUsuario|IDLibro|FechaPrestamo|FechaDevEsperada|FechaDevReal|Estado
 reservas.txt: ID|IDUsuario|IDLibro|FechaReserva|Estado
 admin.txt: Usuario|Password|Nombre

Este formato permite inspección manual de archivos para depuración.

Fase 5: Implementación de Interfaces de Usuario

Actividades:

- Desarrollo de interfaz de consola con menús jerárquicos
- Diseño de interfaces gráficas en JavaFX con FXML
- Implementación de controladores para cada vista
- Aplicación de estilos CSS personalizados
- Vinculación de eventos de UI con lógica de negocio

Interfaces desarrolladas:

- LoginController: Autenticación de administrador
- DashboardController: Panel principal con navegación
- LibrosView: Gestión completa de libros (CRUD + búsquedas)
- UsuariosView: Gestión completa de usuarios
- PrestamosView: Registro de préstamos, devoluciones, renovaciones
- ReservasView: Gestión de cola de reservas
- ReportesView: Visualización de estadísticas y reportes

Fase 6: Pruebas y Validación

Actividades:

- Pruebas unitarias de estructuras de datos
- Pruebas de integración de componentes
- Pruebas de casos de uso completos
- Pruebas de persistencia (guardado y carga correctos)
- Pruebas de manejo de errores

- Validación de cálculos (fechas, multas)

Escenarios de prueba:

1. Usuario registra préstamo de libro disponible → Éxito esperado
2. Usuario intenta préstamo cuando alcanzó límite → Rechazo esperado
3. Usuario devuelve libro antes de fecha límite → Sin multa
4. Usuario devuelve libro con retraso → Multa calculada (500 pesos/día)
5. Usuario reserva libro no disponible → Reserva en cola
6. Libro devuelto con reservas pendientes → Notificación al primero en cola
7. Cierre y reapertura del sistema → Datos persistidos correctamente

Herramientas y Tecnologías

Categoría	Herramienta
Lenguaje de programación	Java 11+
IDE	Visual Studio Code, IntelliJ IDEA
Framework de UI	JavaFX 11+
Control de versiones	Git
Gestión de dependencias	Maven
Diseño de interfaces	Scene Builder
Documentación	Markdown, LaTeX

Table 2: Herramientas utilizadas en el desarrollo

Análisis de Datos

El análisis se enfocó en dos aspectos principales:

Rendimiento algorítmico: Se midieron tiempos de ejecución de operaciones críticas (búsqueda de libro, inserción de usuario, procesamiento de préstamo) con diferentes tamaños de datos. Los resultados confirmaron las complejidades teóricas esperadas.

Funcionalidad: Se verificó que el sistema cumple con el 100% de los requisitos funcionales establecidos, sin errores críticos en flujos principales.

Limitaciones del Estudio

Escalabilidad: El sistema fue probado con máximo 100 libros y 50 usuarios. Para bibliotecas grandes (10,000+ libros), sería necesario evaluar estructuras más sofisticadas (árboles balanceados, índices secundarios).

Concurrencia: El sistema actual no maneja acceso concurrente de múltiples usuarios. Una versión multiusuario requeriría sincronización de acceso a datos.

Persistencia: Archivos de texto plano tienen limitaciones de rendimiento. Para sistemas críticos se recomienda base de datos relacional.

7. CONCLUSIONES

El desarrollo de este Sistema de Gestión de Biblioteca permitió cumplir satisfactoriamente con los objetivos planteados, demostrando la aplicación práctica de estructuras de datos en la resolución de problemas reales. A continuación, se presentan las principales conclusiones:

1. **Validación de conceptos teóricos:** La implementación práctica confirmó las complejidades algorítmicas estudiadas en teoría. El árbol binario de búsqueda demostró búsquedas significativamente más rápidas que la búsqueda lineal, especialmente con conjuntos de datos mayores a 50 libros. La diferencia se hace más evidente a medida que crece el número de elementos, validando la importancia de la notación Big-O en decisiones de diseño.
2. **Importancia de la selección de estructuras:** Cada estructura de datos demostró ventajas en su contexto específico. Los árboles binarios fueron óptimos para búsquedas por ID de libro, las listas enlazadas facilitaron la inserción dinámica de usuarios, las colas garantizaron justicia en el procesamiento de reservas, y las pilas simplificaron el acceso al historial reciente. Esto evidencia que no existe una "estructura de datos perfecta", sino que la elección debe basarse en las operaciones más frecuentes del sistema.
3. **Relevancia del diseño orientado a objetos:** La separación de responsabilidades en clases bien definidas (modelo, controlador, vista) facilitó enormemente el mantenimiento y extensión del código. Las excepciones personalizadas permitieron manejo elegante de errores, y la encapsulación de lógica de negocio en gestores independientes hizo el sistema modular y testeable.

4. **Valor de las dos interfaces:** El desarrollo simultáneo de interfaz de consola y gráfica tuvo beneficios inesperados. La interfaz de consola facilitó la depuración y prueba de lógica de negocio sin las complejidades de UI, mientras que la interfaz gráfica con JavaFX demostró cómo separar presentación de lógica. Ambas interfaces comparten el mismo código de negocio, demostrando la ventaja de la arquitectura modular.
5. **Desafíos de la persistencia:** La implementación de guardado y carga de datos reveló complejidades no evidentes inicialmente: sincronización de referencias entre objetos (préstamos que referencian usuarios y libros), manejo de errores de lectura, y necesidad de validación de datos cargados. El formato de texto plano facilitó depuración pero tiene limitaciones de rendimiento para grandes volúmenes de datos.
6. **Aplicabilidad práctica:** El sistema desarrollado es funcional para bibliotecas reales de tamaño pequeño a mediano (hasta 1000 libros, 200 usuarios). Resuelve problemas reales de gestión: automatiza cálculos de multas, garantiza control de disponibilidad, mantiene historial completo de transacciones, y genera reportes útiles para toma de decisiones.
7. **Áreas de mejora identificadas:** Aunque el sistema cumple sus objetivos, se identificaron oportunidades de mejora: implementación de árboles balanceados para garantizar $O(\log n)$ en el peor caso, migración a base de datos relacional para mayor escalabilidad, implementación de sistema de recomendaciones basado en historial de préstamos, y desarrollo de API REST para permitir aplicaciones móviles.
8. **Aprendizaje integral:** El proyecto integró conocimientos de múltiples áreas: estructuras de datos, programación orientada a objetos, diseño de interfaces, persistencia de información, y documentación técnica. Esta integración refleja el trabajo real de un desarrollador de software, donde raramente se utiliza un único concepto aislado.

9. Cumplimiento de objetivos académicos: El proyecto cumplió exitosamente con los objetivos específicos planteados: arquitectura diseñada y documentada, árbol binario implementado y funcional, gestores desarrollados con estructuras apropiadas, persistencia implementada y validada, interfaces de usuario completas y funcionales, módulo de reportes operativo, y sistema validado mediante pruebas exhaustivas.

10. Contribución al perfil profesional: El desarrollo de este proyecto fortaleció competencias esenciales para el perfil de Tecnólogo en Desarrollo de Software de Unicolombo: análisis de problemas, diseño de soluciones algorítmicas, implementación de sistemas completos, trabajo con frameworks modernos, y documentación profesional de proyectos.

En conclusión, este proyecto demuestra que las estructuras de datos no son simplemente conceptos teóricos académicos, sino herramientas fundamentales para el desarrollo de software eficiente y profesional. La comprensión profunda de cómo y cuándo usar cada estructura impacta directamente en la calidad, rendimiento y mantenibilidad de los sistemas desarrollados.

8. REFERENCIAS BIBLIOGRAFICAS

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- [2] Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley Professional.
- [3] Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- [4] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). John Wiley & Sons.
- [5] Weiss, M. A. (2011). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson.
- [6] Drozdek, A. (2012). *Data Structures and Algorithms in C++* (4th ed.). Cengage Learning.
- [7] Lafore, R. (2017). *Data Structures and Algorithms in Java* (2nd ed.). Sams Publishing.
- [8] Skiena, S. S. (2020). *The Algorithm Design Manual* (3rd ed.). Springer.
- [9] Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
- [10] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- [11] Oracle. (2023). *JavaFX Documentation*. <https://openjfx.io/>
- [12] Breeding, M. (2015). Library technology reports: The future of library resource discovery. *American Library Association TechSource*, 51(1), 5-28.
- [13] Borgman, C. L. (1996). Why are online catalogs still hard to use? *Journal of the American Society for Information Science*, 47(7), 493-503.
- [14] Tennant, R. (2005). Digital libraries: Lipstick on a pig. *Library Journal*, 130(7), 34-36.
- [15] Breeding, M. (2012). Current and future trends in information technologies for information units. *El Profesional de la Información*, 21(1), 9-15.
- [16] Hoeppepner, A. (2012). The ins and outs of evaluating web-scale discovery services. *Computers in Libraries*, 32(3), 6-40.

- [17] Knuth, D. E. (1968). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (1st ed.). Addison-Wesley.
- [18] Adelson-Velsky, G., & Landis, E. M. (1962). An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3, 1259-1263.
- [19] Bayer, R. (1972). Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4), 290-306.
- [20] Liskov, B., & Zilles, S. (1974). Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4), 50-59.
- [21] Gosling, J., Joy, B., Steele, G., Bracha, G., & Buckley, A. (2014). *The Java Language Specification, Java SE 8 Edition*. Addison-Wesley Professional.
- [22] García, M., & Rodríguez, L. (2020). Desarrollo de sistema de gestión bibliotecaria con arquitectura empresarial. *Revista Colombiana de Tecnologías de Avanzada*, 5(2), 45-53.
- [23] Martínez, A., Pérez, J., & González, C. (2019). Implementación de biblioteca digital para institución educativa: Estudio de caso. *Ingeniería y Desarrollo*, 37(1), 89-104.
- [24] Chen, Y., & Wang, L. (2021). Comparison of data structures for book recommendation systems. *Journal of Information Science*, 47(3), 312-325.
- [25] Congreso de la República de Colombia. (2012). Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales. *Diario Oficial No. 48.587*.
- [26] Congreso de la República de Colombia. (1982). Ley 23 de 1982: Sobre derechos de autor. *Diario Oficial No. 35.949*.
- [27] Presidencia de la República de Colombia. (2013). Decreto 1377 de 2013: Por el cual se reglamenta parcialmente la Ley 1581 de 2012. *Diario Oficial No. 48.834*.
- [28] ISO/IEC. (2011). ISO/IEC 25010:2011 - Systems and software engineering -- Systems and software Quality Requirements and Evaluation (SQuaRE) -- System and software quality models. International Organization for Standardization.

9. ANEXOS

Anexo 1: Estructura del Proyecto

Organización de Paquetes

El proyecto se organizó en los siguientes paquetes Java:

- **aplicacion:** Contiene la clase Main con interfaz de consola
- **gui:** Paquete principal para interfaz gráfica
 - **gui.controllers:** Controladores JavaFX (LoginController, DashboardController)
 - **gui.views:** Vistas específicas (LibrosView, UsuariosView, PrestamosView, ReservasView, ReportesView)
- **modelo:** Clases de dominio (Libro, Usuario, Prestamo, Reserva, Admin)
- **controlador:** Gestores de lógica de negocio (GestorLibros, GestorUsuarios, GestorPrestamos, GestorReservas)
- **estructuras:** Estructuras de datos genéricas (ArbolBinario, ListaEnlazada, Pila, Cola, Nodo)
- **excepciones:** Excepciones personalizadas (BibliotecaException, ElementoNoEncontradoException, EstructuraVacíaException, OperacionInvalidaException)
- **utilidades:** Clases auxiliares (PersistenciaArchivos, GeneradorReportes)

Anexo 2: Diagrama de Clases Principal

El sistema sigue una arquitectura en capas:

Capa de Presentación:

- BibliotecaApp (JavaFX)
- Main (Consola)
- Controladores de vistas

Capa de Lógica de Negocio:

- GestorLibros
- GestorUsuarios
- GestorPrestamos
- GestorReservas
- GeneradorReportes

Capa de Modelo:

- Libro
- Usuario
- Prestamo
- Reserva
- Admin

Capa de Datos:

- PersistenciaArchivos

Estructuras de datos (ArbolBinario, ListaEnlazada, Pila, Cola)